

Algoritmusok és adatszerkezetek II. előadásjegyzet: Fák

Ásványi Tibor – asvanyi@inf.elte.hu

2024. október 25.

Tartalomjegyzék

1. AVL fák	4
1.1. AVL fák: beszúrás	9
1.2. AVL fák: a $\text{remMin}(t, \text{minp})$ eljárás	14
1.3. AVL fák: törlés	16
1.4. Az AVL fák magassága*	18
2. Általános fák	21
3. B+ fák és műveleteik	23

Hivatkozások

- [1] ÁSVÁNYI TIBOR, Algoritmusok és adatszerkezetek II.
Útmutatások a tanuláshoz, Tematika, fák, gráfok, mintaillesztés, tömörítés
<http://aszt.inf.elte.hu/~asvanyi/ad/ad2jegyzet/>
- [2] CORMEN, T.H., LEISERSON, C.E., RIVEST, R.L., STEIN, C.,
magyarul: Új Algoritmusok, *Scolar Kiadó*, Budapest, 2003.
ISBN 963 9193 90 9
angolul: Introduction to Algorithms (Third Edititon),
The MIT Press, 2009.
- [3] FEKETE ISTVÁN, Algoritmusok jegyzet
<http://ifekete.web.elte.hu/>
- [4] RÓNYAI LAJOS – IVANYOS GÁBOR – SZABÓ RÉKA, Algoritmusok,
TypoTeX Kiadó, 1999. ISBN 963 9132 16 0
https://www.tankonyvtar.hu/hu/tartalom/tamop425/2011-0001-526_ronyai_algoritmusok/adatok.html
- [5] WEISS, MARK ALLEN, Data Structures and Algorithm Analysis,
Addison-Wesley, 1995, 1997, 2007, 2012, 2013.
- [6] CARL BURCH, ÁSVÁNYI TIBOR, B+ fák
<http://aszt.inf.elte.hu/~asvanyi/ad/ad2jegyzet/B+fa.pdf>
- [7] ÁSVÁNYI TIBOR, Algoritmusok és adatszerkezetek I. előadásjegyzet
(2021)
<http://aszt.inf.elte.hu/~asvanyi/ad/ad1jegyzet/ad1jegyzet.pdf>
- [8] WIRTH, N., Algorithms and Data Structures,
Prentice-Hall Inc., 1976, 1985, 2004.
magyarul: Algoritmusok + Adatstruktúrák = Programok, *Műszaki Könyvkiadó*, Budapest, 1982. ISBN 963 10 3858 0

1. AVL fák

Az előző félévben tanultunk a bináris keresőfákról, mint (a hasító táblák mellett) adathalmazok, szótárak reprezentációjára szolgáló adatszerkezetekről. Láttuk, hogy a tipikus szótárműveletek (mint egy kulcs – és a hozzá tartozó adatok – keresése, beszúrása és eltávolítása) átlagos esetben megfelelő hatékonyságúak, mert a futási idő legfeljebb a fa magasságával arányos, ennek várható értéke pedig $\Theta(\log n)$, ahol n a fa mérete, azaz csúcsainak száma, és a $\log n$ függvény nagyon lassan nő.

A legrosszabb esetben azonban a bináris keresőfa (BST) magassága $n - 1$, így a szótárműveletek sem elég hatékonyak. Ezen segít a kiegyensúlyozott keresőfák alkalmazása, mert ezeknél a fa magassága minden esetben $\Theta(\log n)$, ami – megfelelő implementáció mellett – garantálja a szótárműveletek megfelelő hatékonyságát.

A különböző fajta kiegyensúlyozott bináris keresőfák klasszikus típusai a piros-fekete [2] és az AVL fák. Ez utóbbiakat tárgyaljuk itt. Amennyiben az alkalmazásunkban lényegesen több a szótárban való keresés, mint az azt módosító művelet, az AVL fák alkalmazása előnyösebb, mert a piros-fekete fáknál az AVL fák erősebben kiegyensúlyozottak. Az ideális a majdnem teljes bináris keresőfák alkalmazása lehetne (ui. ezek magassága minimális), de ezek módosítására nem ismerünk hatékony megoldásokat. Az AVL rövidítés mögött *Georgy Adelson-Velsky* és *Eugenii Landis* szovjet matematikusok nevét fedezhetjük fel, akik 1963-ban dolgozták ki koncepciójukat.

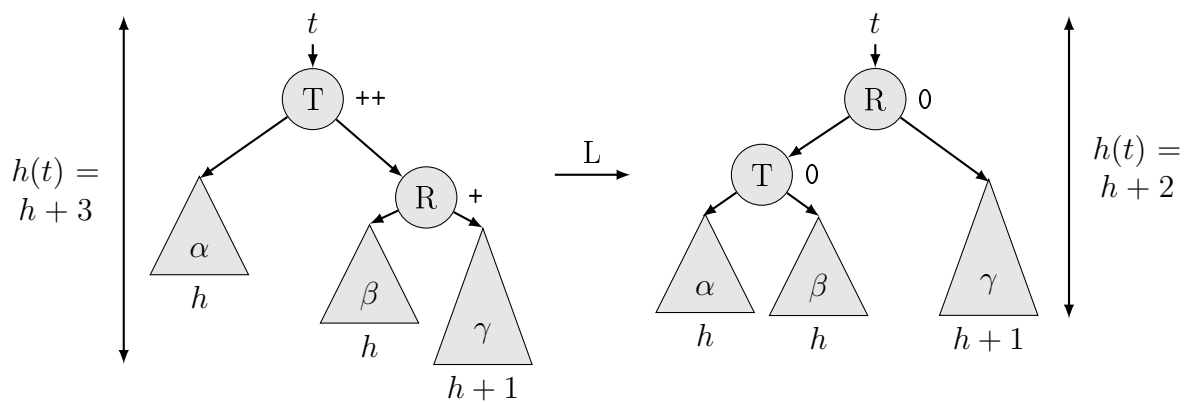
Az AVL fa módosításával járó műveletek, mint pl. a beszúrás és a törlés, elronthatják a kiegyensúlyozottságot, amit – kiegyensúlyozó forgatások segítségével – még a művelet befejezése előtt helyre kell állítani.

A témával ismerősek kedvéért előljáróban megjegyezzük, hogy az AVL fák kiegyensúlyozásának szabályai megtalálhatók az 1-6. ábrákon.

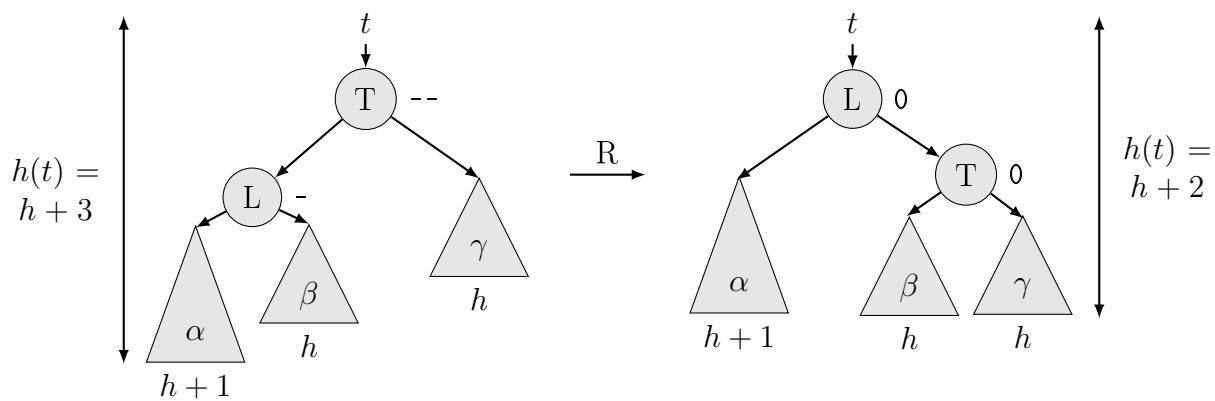
Ebben a fejezetben a továbbiakban részletesen tárgyaljuk az AVL fa fogalmát, tulajdonságait, egy különösen hatékony megvalósítását lehetővé tevő reprezentációját, és ennek megfelelően a műveleteit.

Az *AVL fák* magasság szerint kiegyensúlyozott bináris keresőfák (height-balanced BSTs). Egy bináris fa *magasság szerint kiegyensúlyozott*, ha minden csúcsa kiegyensúlyozott. (Mostantól *kiegyensúlyozott* alatt *magasság szerint kiegyensúlyozottat* értünk.) Egy bináris fa egy $(*p)$ csúcsa *kiegyensúlyozott* (balanced node), ha a csúcs $p \rightarrow b$ egyensúlyára $|p \rightarrow b| \leq 1$. A $(*p)$ csúcs egyensúlya [the $(*p)$ node's balance] definíció szerint:

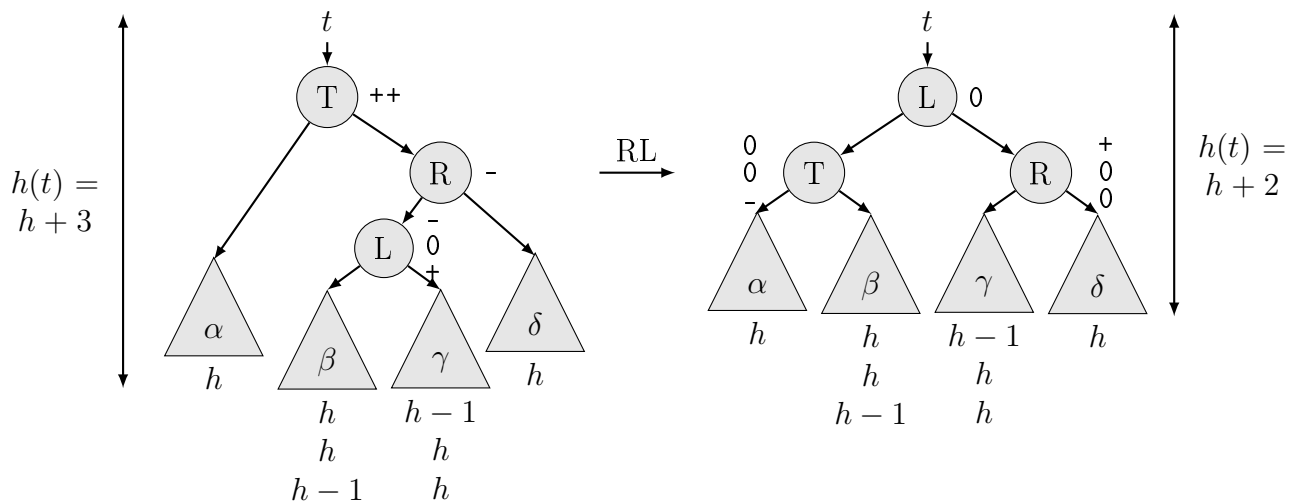
$$p \rightarrow b = h(p \rightarrow \text{right}) - h(p \rightarrow \text{left})$$



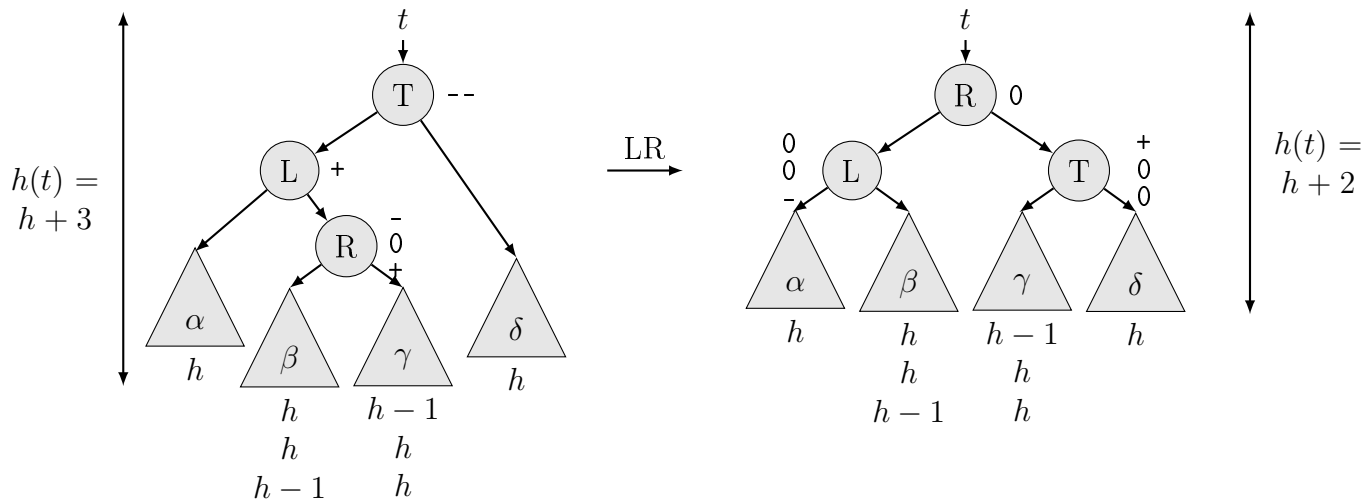
1. ábra. $(++, +)$ forgatás.



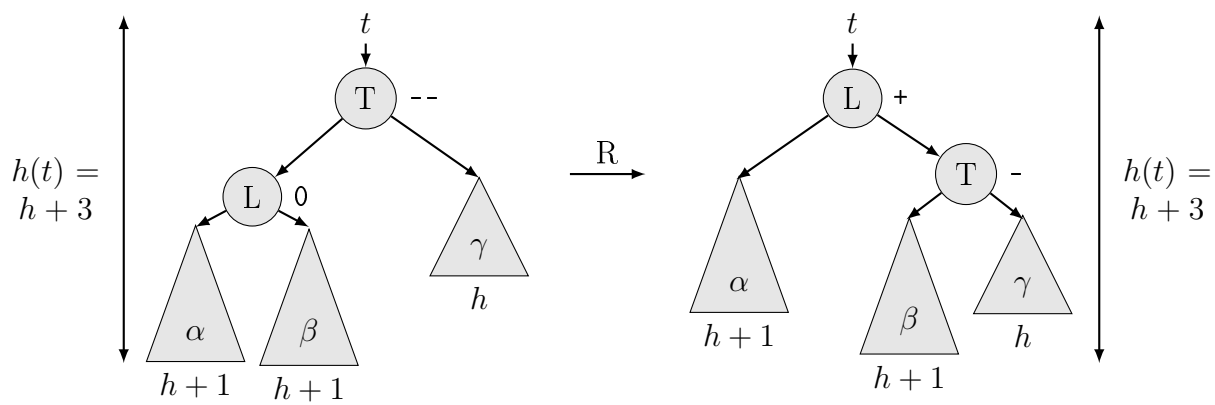
2. ábra. $(--, -)$ forgatás.



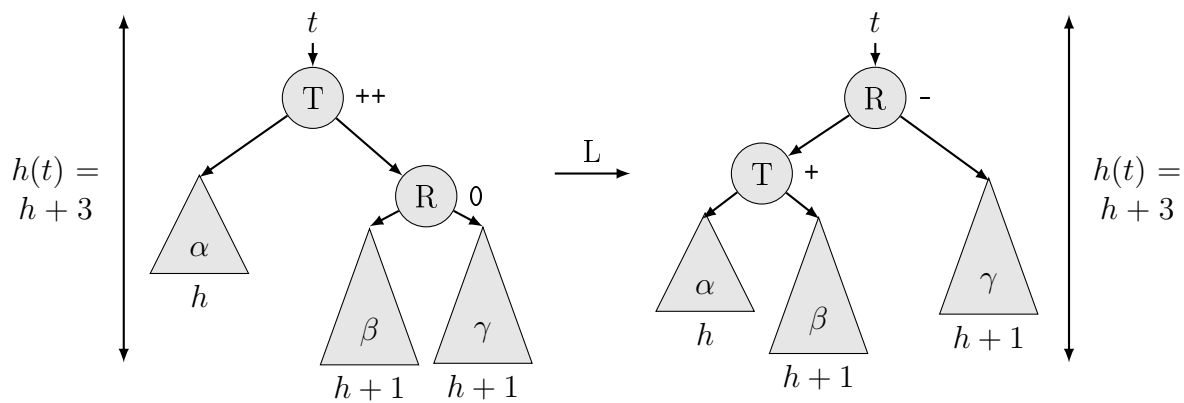
3. ábra. $(++, -)$ forgatás.



4. ábra. $(--, +)$ forgatás.



5. ábra. $(--, 0)$ forgatás.



6. ábra. $(++, 0)$ forgatás.

Az AVL fákat láncoltan reprezentáljuk. A csúcsokban a b egyensúly attribútumot expliciten tároljuk. (Egy másik lehetőség a két részfa magasságainak tárolása lenne; egy harmadik, hogy csak az aktuális részfa magasságát tároljuk.) A Node osztályt tehát a következőképpen módosítjuk:

Node
+ $key : \mathcal{T}$ // $\mathcal{T}$ is some known type
+ $b : -1..1$ // the balance of the node
+ $left, right : \text{Node}^*$
+ $\text{Node}() \{ left := right := \odot ; b := 0 \}$ // create a tree of a single node
+ $\text{Node}(x:\mathcal{T}) \{ left := right := \odot ; b := 0 ; key := x \}$

Egyelőre részletes bizonyítás nélkül közöljük az alábbi eredményt:

Tétel: Tetszőleges n csúcsú nemüres AVL fa h magasságára:

$$\lfloor \log n \rfloor \leq h \leq 1,45 \log n, \quad \text{azaz} \quad h \in \Theta(\log n)$$

A bizonyítás vázlata: Először a h magasságú, nemüres KBF-ek (kiegyensúlyozott, bináris fák) n méretére adunk alsó és felső becslést. Az $n < 2^{h+1}$ becslésből azonnal adódik $\lfloor \log n \rfloor \leq h$. Másrészt meghatározzuk a h mélységű, legkisebb méretű KBF-ek csúcsainak f_h számát. Erre kapjuk, hogy $f_0 = 1, f_1 = 2, f_h = 1 + f_{h-1} + f_{h-2} \quad (h \geq 2)$. Ezért az ilyen fákat *Fibonacci fák*nak hívjuk. Mivel tetszőleges h magasságú KBF n méretére $n \geq f_h$, némi matematikai ügyességgel kaphatjuk a $h \leq 1,45 \log n$ egyenlőtlenséget.

Mivel az AVL fák magassága $\Theta(\log n)$, ezért a bináris keresőfák $\text{search}(t, k)$, $\text{min}(t)$ és $\text{max}(t)$ függvényeire t AVL fa esetén automatikusan $MT(n) \in \Theta(\log n)$, ahol $n = |t|$.

Az $\text{insert}(t, k)$, a $\text{del}(t, k)$, a $\text{remMin}(t, \text{minp})$ és a $\text{remMax}(t, \text{maxp})$ eljárások azonban változtatják a fa alakját. Így elromolhat a kiegyensúlyozottság, és már nem garantált a fenti műveletigény. Ennek elkerülésére ezeket az eljárásokat úgy módosítjuk, hogy minden egyes rekurzív eljáráshívás után ellenőrizni fogjuk, hogyan változott a megfelelő részfa magassága, és ez hogyan befolyásolta a felette levő csúcs kiegyensúlyozottságát, szükség esetén helyreállítva azt. Ez minden szinten legfeljebb konstans mennyiségű extra műveletet fog jelenteni, és így a kiegészített eljárások futási ideje megtartja az $MT(n) \in \Theta(\log n)$ (ahol $n = |t|$) nagyságrendet.

A példákhoz a $(\text{bal_részfa gyökér jobb_részfa})$ jelölést használjuk, ahol az üres (rész)fákat üres zárójelpár jelöli. A könnyebb olvashatóság kedvéért [],

$\{\}$ és esetleg $\langle \rangle$ zárójeleket is alkalmazunk, a levécsúcsoknál pedig a két üres részfát nem jelöljük.

Például a $\{[2]4[(6)8(10)]\}$ bináris keresőfa gyökere a 4; bal részfája a $[2]$, ami egyetlen levécsúcsból (és annak üres bal és jobb részfaiból) áll; jobb részfája a $[(6)8(10)]$, aminek gyökere a 8, bal részfája a (6) , jobb részfája a (10) . Az így ábrázolt fák inorder bejárása a zárójelek elhagyásával adódik. A belső csúcsok egyensúlyait kb formában jelöljük, ahol k a csúcsot azonosító kulcs, b pedig a csúcs egyensúlya, a $0:\circ$, $1:+$, $2:++$, $-1:-$, $-2:--$ megfeleltetéssel. Mivel a levécsúcsok egyensúlya mindig nulla, a leveleknél nem jelöljük az egyensúlyt. A fenti AVL fa például a megfelelő egyensúlyokkal a következő: $\{[2]4+[(6)8\circ(10)]\}$

Az AVL fák műveletei során a kiegyensúlyozatlan részfák kiegyensúlyozásához *forgatásokat* fogunk használni. Az alábbi forgatási sémákban görög kisbetűk jelölik a részfákat (amelyek minden esetben AVL fák lesznek), latin nagybetűk pedig a csúcsok kulcsait (1. és 2. ábrák, az egyensúlyok nélkül):

Balra forgatás (Left rotation): $[\alpha \text{ T } (\beta \text{ R } \gamma)] \rightarrow [(\alpha \text{ T } \beta) \text{ R } \gamma]$

Jobbra forgatás (Right rotation): $[(\alpha \text{ L } \beta) \text{ T } \gamma] \rightarrow [\alpha \text{ L } (\beta \text{ T } \gamma)]$

Vegyük észre, hogy a fa inorder bejárását egyik forgatás sem változtatja, így a bináris keresőfa tulajdonságot is megtartják. Mint látni fogjuk, a kiegyensúlyozatlan részfák kiegyensúlyozása minden esetben egy vagy két forgatásból áll. Ezért a bináris keresőfa tulajdonságot a kiegyensúlyozások is megtartják.

1.1. AVL fák: beszúrás

Nézzük most először a bináris keresőfák $\text{insert}(t, k)$ eljárását!

A $\{[2]4+[(6)8\circ(10)]\}$ AVL fából az 1 beszúrásával az $\{[(1)2-(\circ)]4\circ[(6)8\circ(10)]\}$ AVL fa adódik, a 3 beszúrásával pedig az $\{[(\circ)2+(3)]4\circ[(6)8\circ(10)]\}$ AVL fa.

A fenti $\{[2]4+[(6)8\circ(10)]\}$ AVL fából a 7 beszúrásával azonban a $\{[2]4++[(\langle 6+\langle 7 \rangle)8-(10)]\}$ fát kapjuk, a 9 beszúrásával pedig a $\{[2]4++[(6)8+(\langle 9 \rangle 10-\langle \rangle)]\}$ fát. Mindkettő bináris keresőfa, de már nem AVL fa, mivel a $4++$ csúcs nem kiegyensúlyozott.

A legutolsó esetben a bináris keresőfa könnyen kiegyensúlyozható, ha megfontoljuk, hogy az a következő sémára illeszkedik, amiben görög kisbetűk jelölik a kiegyensúlyozott részfákat, latin nagybetűk pedig a csúcsok kulcsait:

$[\alpha \text{ T}++ (\beta \text{ R}+ \gamma)]$, ahol $\text{T}=4$, $\alpha=[2]$, $\text{R}=8$, $\beta=(6)$ és $\gamma=(\langle 9 \rangle 10-\langle \rangle)$.

A kiegyensúlyozáshoz szükséges transzformáció a fa *balra forgatása* (1. ábra):

$$[\alpha \text{ T}++ (\beta \text{ R}+ \gamma)] \rightarrow [(\alpha \text{ T} \circ \beta) \text{ R} \circ \gamma]$$

A fenti példán ez a következőt jelenti:

$$\{ [2]4++[(6)8+(\langle 9 \rangle 10-\langle \rangle)] \} \rightarrow \{ [(2)4 \circ (6)] 8 \circ [(9)10-\langle \rangle] \}$$

A transzformáció helyességének belátásához bevezetjük a $h = h(\alpha)$ jelölést. Ebből a kiinduló fára a T++ és R+ egyensúlyok miatt $h((\beta \text{ R} \gamma)) = h+2$, $h(\gamma) = h+1$ és $h(\beta) = h$ adódik, innét pedig az eredmény fára $h(\alpha) = h = h(\beta)$ miatt T $\circ$; továbbá $h((\alpha \text{ T} \circ \beta)) = h+1 = h(\gamma)$ miatt R $\circ$ adódik.

Az eredeti $\{ [2]4+[(6)8 \circ (10)] \}$ AVL fába a 7 beszúrásával adódó $\{ [2]4++[(\langle \rangle 6 + \langle 7 \rangle)8 - (10)] \}$ kiegyensúlyozatlan bináris keresőfa kiegyensúlyozásával pedig $\{ [(2)4-\langle \rangle] 6 \circ [(7)8 \circ (10)] \}$ adódik, amire az

$$\{ \alpha \text{ T}++ [(\beta \text{ L}- \circ + \gamma) \text{ R}- \delta] \} \rightarrow \{ [\alpha \text{ T} \circ \circ - \beta] \text{ L} \circ [\gamma \text{ R} + \circ \circ \delta] \}$$

séma (3. ábra) szolgál, ahol α , β , γ és δ AVL fák, és a három jelből álló egyensúlyok sorban három esetet jelentenek úgy, hogy a bal oldalon az i -edik alternatívának a jobb oldalon is az i -edik eset felel meg ($i \in \{1; 2; 3\}$).

A fenti séma a példában T=4, $\alpha = [2]$, R=8, $\delta = (10)$, L=6, + egyensúllyal (harmadik eset), $\beta = \circ$ és $\gamma = \langle 7 \rangle$ helyettesítéssel alkalmazható.

A transzformációt *kettős forgatásnak* is tekinthetjük: Az $\{ \alpha \text{ T} [(\beta \text{ L} \gamma) \text{ R} \delta] \}$ fára először az R csúcsnál alkalmaztunk egy jobbra forgatást, aminek eredményeképpen az $\{ \alpha \text{ T} [\beta \text{ L} (\gamma \text{ R} \delta)] \}$ bináris keresőfát kaptuk, majd az eredmény fát balra forgattuk, ami az $\{ [\alpha \text{ T} \beta] \text{ L} [\gamma \text{ R} \delta] \}$ AVL fát eredményezte.

A kettős forgatás helyességének ellenőrzéséhez kiszámítjuk az eredmény fa egyensúlyait. Most is bevezetjük a $h = h(\alpha)$ jelölést. Mivel a kettős forgatás előtti fában T++, azért $h([\beta \text{ L} \gamma) \text{ R} \delta]) = h+2$. Innét R- miatt $h((\beta \text{ L} \gamma)) = h+1$ és $h(\delta) = h$. Most L lehetséges egyensúlyai szerint három eset van.

- (1) L- esetén $h(\beta) = h$ és $h(\gamma) = h-1 \Rightarrow$ az eredmény fában T $\circ$ és R+.
 - (2) L $\circ$ esetén $h(\beta) = h$ és $h(\gamma) = h \Rightarrow$ az eredmény fában T $\circ$ és R $\circ$.
 - (3) L+ esetén $h(\beta) = h-1$ és $h(\gamma) = h \Rightarrow$ az eredmény fában T- és R $\circ$.
- Mindhárom esetben $h([\alpha \text{ T} \beta]) = h+1 = h([\gamma \text{ R} \delta])$, így L $\circ$.

Ezzel beláttuk a kettős forgatási séma helyességét.

Vegyük észre, hogy a fentiek alapján, az L csúcsnak a kettős forgatás előtti egyensúlyát s -sel, a T és a R csúcsoknak pedig a kettős forgatás utáni egyensúlyát rendre s_t -vel, illetve s_r -vel jelölve a következő összefüggéseket írhatjuk fel:

$$s_t = -\lfloor (s+1)/2 \rfloor \quad \text{és} \quad s_r = \lfloor (1-s)/2 \rfloor$$

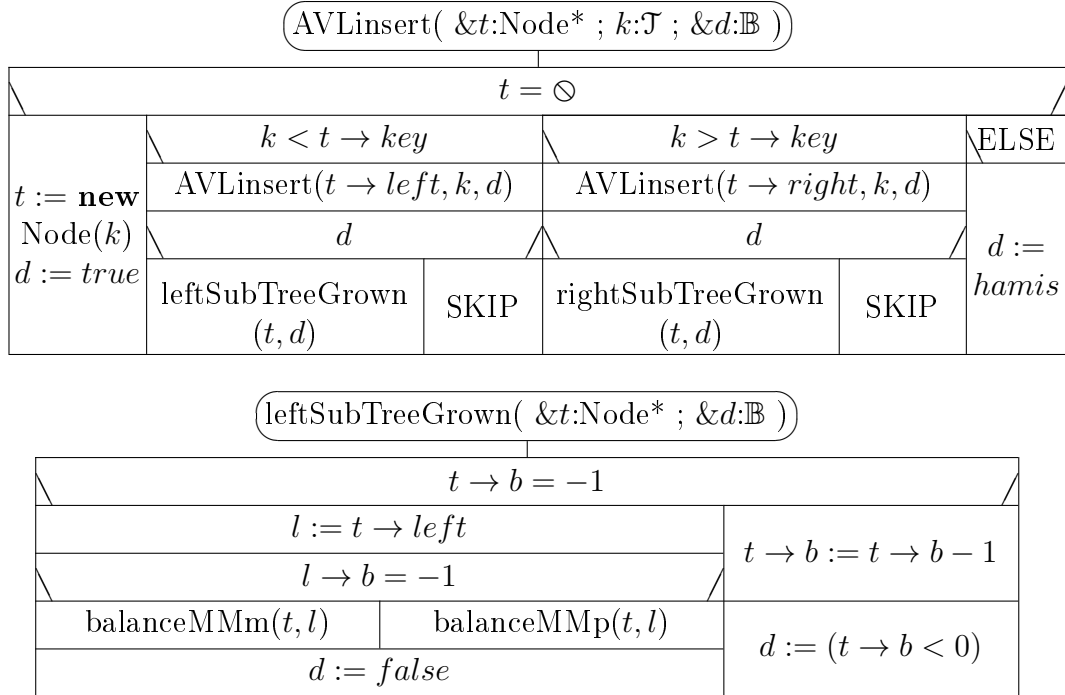
Az eddigi két kiegyensúlyozási séma mellett természetes módon adódnak ezek tükörképei, a forgatások előtt a T— csúccsal a gyökérben. Ezek tehát a következők (2. és 4. ábra):

$$\begin{aligned} & [(\alpha \text{ L} - \beta) \text{ T} - - \gamma] \rightarrow [\alpha \text{ L} \circ (\beta \text{ T} \circ \gamma)] \\ & \{ [\alpha \text{ L} + (\beta \text{ R} - \circ + \gamma)] \text{ T} - - \delta \} \rightarrow [(\alpha \text{ L} \circ \circ - \beta) \text{ R} \circ (\gamma \text{ T} + \circ \circ \delta)] \end{aligned}$$

$$s_l = -\lfloor (s+1)/2 \rfloor \quad \text{és} \quad s_t = \lfloor (1-s)/2 \rfloor$$

ahol s az R csúcs kettős forgatás előtti egyensúlya; s_l , illetve s_t pedig rendre az L és T csúcsoknak a kettős forgatás utáni egyensúlya

Eddig nem beszéltünk arról, hogy az AVL fában az új csúcs beszúrása után mely csúcsoknak és milyen sorrendben számoljuk újra az egyensúlyát, sem hogy mikor ütemezzük be a kiegyensúlyozást. Csak a beszúrás nyomvonalán visszafelé haladva kell újraszámolnunk az egyensúlyokat, és csak itt kell kiegyensúlyoznunk, ha kiegyensúlyozatlan csúcsot találunk. Így a futási idő a fa magasságával arányos marad, ami AVL fákra $O(\log n)$. Most tehát részletezzük a beszűrő és kiegyensúlyozó algoritmus működését. Ennek során a fa magassága vagy eggyel növekszik ($d = \text{true}$), vagy ugyanannyi marad ($d = \text{false}$).



rightSubTreeGrown(&t:Node* ; &d:ℤ)		
$t \rightarrow b = 1$		
$r := t \rightarrow right$		$t \rightarrow b := t \rightarrow b + 1$
$r \rightarrow b = 1$		
balancePPp(t, r)	balancePPm(t, r)	$d := (t \rightarrow b > 0)$
$d := false$		

balancePPp(&t, r : Node*)

$t \rightarrow right := r \rightarrow left$
$r \rightarrow left := t$
$r \rightarrow b := t \rightarrow b := 0$
$t := r$

balancePPm(&t, r : Node*)

$l := r \rightarrow left$
$t \rightarrow right := l \rightarrow left$
$r \rightarrow left := l \rightarrow right$
$l \rightarrow left := t$
$l \rightarrow right := r$
$t \rightarrow b := -\lfloor (l \rightarrow b + 1)/2 \rfloor$
$r \rightarrow b := \lfloor (1 - l \rightarrow b)/2 \rfloor$
$l \rightarrow b := 0$
$t := l$

balanceMMm(&t, l : Node*)

$t \rightarrow left := l \rightarrow right$
$l \rightarrow right := t$
$l \rightarrow b := t \rightarrow b := 0$
$t := l$

balanceMMp(&t, l : Node*)

$r := l \rightarrow right$
$l \rightarrow right := r \rightarrow left$
$t \rightarrow left := r \rightarrow right$
$r \rightarrow left := l$
$r \rightarrow right := t$
$l \rightarrow b := -\lfloor (r \rightarrow b + 1)/2 \rfloor$
$t \rightarrow b := \lfloor (1 - r \rightarrow b)/2 \rfloor$
$r \rightarrow b := 0$
$t := r$

Az AVL fába való beszúrást röviden összefoglalva:

1. Megkeressük a kulcs helyét a fában.
2. Ha a kulcs benne van a fában, STOP.
3. Ha a kulcs helyén egy üres részfa található, beszúrunk az üres fa helyére egy új, a kulcsot tartalmazó levélcsúcsot. Ez a részfa eggyel magasabb lett.
4. Ha a gyökércsúcsnál vagyunk, STOP. Különben egyet fölfelé lépünk a keresőfában. Mivel az a részfa, amiből fölfelé léptünk, eggyel magasabb lett, az aktuális csúcs egyensúlyát megfelelőképp módosítjuk. (Ha a jobb részfa lett magasabb, hozzáadunk az egyensúlyhoz egyet, ha a bal, levonunk belőle egyet.)
5. Ha az aktuális csúcs egyensúlya 0 lett, akkor az aktuális csúcshoz tartozó

részfa alacsonyabb ága hozzáanótt a magasabbikhoz, tehát az aktuális részfa most ugyanolyan magas, mint a beszúrás előtt volt, és így egyetlen más csúcs egyensúlyát sem kell módosítani: STOP.

6. Ha az aktuális csúcs új egyensúlya 1 vagy -1, akkor előtte 0 volt, ezért az aktuális részfa magasabb lett eggyel. Ekkor a 4. ponttól folytatjuk.

7. Ha az aktuális csúcs új egyensúlya 2 vagy -2¹, akkor a hozzá tartozó részfat ki kell egyensúlyozni. A *kiegyensúlyozás után az aktuális részfa visszanyeri a beszúrás előtti magasságát*, ezért már egyetlen más csúcs egyensúlyát sem kell módosítani: STOP.

Az az állítás, hogy ebben az algoritmusban a *kiegyensúlyozás után az aktuális részfa visszanyeri a beszúrás előtti magasságát*, még igazolásra vár. Azt az esetet nézzük meg, amikor a kiegyensúlyozandó részfa gyökere T++. A T— eset hasonlóan fontolható meg. A T++ esethez tartozó kiegyensúlyozási sémák (1. és 3. ábra):

$$\begin{aligned} & [\alpha \text{ T}++ (\beta \text{ R}+ \gamma)] \rightarrow [(\alpha \text{ T} \circ \beta) \text{ R} \circ \gamma] \\ & \{ \alpha \text{ T}++ [(\beta \text{ L} \circ \circ + \gamma) \text{ R} - \delta] \} \rightarrow \{ [\alpha \text{ T} \circ \circ - \beta] \text{ L} \circ [\gamma \text{ R} + \circ \circ \delta] \} \end{aligned}$$

Először belátjuk, hogy ha a T csúcs jobboldali R gyereke + vagy – súlyú, akkor a fenti sémák közül a megfelelő alkalmazható: Mivel a beszűrő algoritmus a fában a beszúrás helyétől egyesével fölfele lépked, és az első kiegyensúlyozatlan csúcsnál azonnal kiegyensúlyoz, ez alatt nincs kiegyensúlyozatlan csúcs, azaz az α , β és γ , illetve a δ részfák is kiegyensúlyozottak, ez pedig éppen a fenti forgatások feltétele, amellet, hogy bináris keresőfát akarunk kiegyensúlyozni, amit viszont a kiegyensúlyozás nélküli beszűrő algoritmus garantál.

Most még be kell látni, hogy a fenti sémák minden esetet lefednek, azaz R+ vagy R–: egyrészt, R nem lehet a beszúrás által létrehozott új csúcs, mert különben T-nek a beszúrás előtti jobboldali részfája üres lett volna, tehát most nem lehetne T++. Másrészt, ha a fölfele lépkedés során nulla egyensúlyú csúcs áll elő, akkor a fölötte levő csúcsok egyensúlya már nem módosul, így kiegyensúlyozatlan csúcs sem állhat elő. Márpedig most T++. Így tehát az új csúcstól T-ig fölfelé vezető úton minden csúcs, azaz R egyensúlya is + vagy –.

Most belátjuk, hogy a kiegyensúlyozások visszaállítják a részfa beszúrás előtti magasságát. A T++ kiegyensúlyozatlan csúcs a beszúrás előtt kiegyensúlyozott volt. Mivel a beszúrás, a beszúrás helyétől kezdve T-ig fölfelé vezető úton mindegyik részfa magasságát pontosan eggyel növelte, a beszúrás előtt

¹A 2 és -2 eseteket a struktogramban nem számoltuk ki expliciten, hogy az egyensúly tárolására elég legyen két bit.

T+ volt. Ezért a beszúrás előtt, $h = h(\alpha)$ jelöléssel, a T gyökerű részfa $h + 2$ magas volt. A beszúrás után tehát T++ lett, és így a T gyökerű részfa $h + 3$ magas lett. A beszúrás utáni, de még a kiegyensúlyozás előtti állapotot tekintve a továbbiakban megkülönböztetjük a T jobboldali R gyerekére a R+ és a R− eseteket.

R+ esetén már láttuk, hogy $h(\alpha) = h = h(\beta)$ és $h(\gamma) = h + 1$. Ezért a kiegyensúlyozás után $h([\alpha \text{ T } \beta] \text{ R } \gamma) = h + 2$.

R− esetén pedig már láttuk, hogy $h(\alpha) = h = h(\delta)$ és $h([\beta \text{ L } \gamma]) = h + 1$. Ezért $h(\beta), h(\gamma) \leq h$. Így a kiegyensúlyozás után $h(\{[\alpha \text{ T } \beta] \text{ L } [\gamma \text{ R } \delta]\}) = h + 2$.

Ezzel beláttuk, hogy a kiegyensúlyozások mindkét esetben visszaállítják a részfa beszúrás előtti magasságát, mégpedig úgy, hogy a beszúrás által eggyel megnövelt magasságot eggyel csökkentik. Szimmetria okokból ez hasonlóan látható be T−− esetén is, figyelembe véve a L− és a L+ eseteket.

1.2. AVL fák: a remMin($t, minp$) eljárás

Bináris keresőfákra a remMin eljárás:

remMin(& t , & $minp$: Node*)	
$t \rightarrow left = \oslash$	
$minp := t$	remMin($t \rightarrow left, minp$)
$t := minp \rightarrow right$	
$minp \rightarrow right := \oslash$	

Ezt például a $\{ [2]4+[(6)8\circ(10)] \}$ AVL fára alkalmazva, a $minp \rightarrow key = 2$ eredmény mellett, a $\{ [] \text{ } 4++[(6)8\circ(10)] \}$ kiegyensúlyozatlan bináris keresőfa adódna. Látjuk, hogy a kiegyensúlyozatlan 4++ csúcs jobboldali gyereke a 8◦. Ennek az esetnek a kezelésére új kiegyensúlyozási sémát kell alkalmaznunk. Szerencsére egy balra forgatás, a megfelelő egyensúly beállítások mellett most is megoldja a problémát (6. ábra):

$$\{ \alpha \text{ T}++[\beta \text{ R}\circ \gamma] \} \rightarrow \{ [\alpha \text{ T}+ \beta] \text{ R}- \gamma \}.$$

T++ miatt $h = h(\alpha)$ jelöléssel nyilván $h(\beta) = h + 1 = h(\gamma)$, így a forgatás után az egyensúlyokat a fenti séma szerint kell beállítani. A fa magassága a forgatás előtt és után is $h + 3$ lesz, ez a fajta kiegyensúlyozás tehát az eddigiekkel szemben *nem* csökkenti az aktuális részfa magasságát.

Ezután az AVL fákra alkalmazott, a megfelelő kiegyensúlyozásokkal kiegészített AVLremMin eljárás pl. a következő lehet (d most azt jelöli, hogy csökkent-e az aktuális részfa magassága):

AVLremMin($\&t, \&minp:\text{Node}^* ; \&d:\mathbb{B}$)		
$t \rightarrow left = \oslash$		
$minp := t$	AVLremMin($t \rightarrow left, minp, d$)	
$t := minp \rightarrow right$		
$minp \rightarrow right := \oslash$	d	
$d := true$	leftSubTreeShrunk(t, d)	SKIP

leftSubTreeShrunk($\&t:\text{Node}^* ; \&d:\mathbb{B}$)	
$t \rightarrow b = 1$	
balance_PP(t, d)	$t \rightarrow b := t \rightarrow b + 1$
	$d := (t \rightarrow b = 0)$

balance_PP($\&t:\text{Node}^* ; \&d:\mathbb{B}$)		
$r := t \rightarrow right$		
$r \rightarrow b = -1$	$r \rightarrow b = 0$	$r \rightarrow b = 1$
balancePPm(t, r)	balancePP0(t, r)	balancePPp(t, r)
	$d := false$	

balancePP0($\&t, r : \text{Node}^*$)
$t \rightarrow right := r \rightarrow left$
$r \rightarrow left := t$
$t \rightarrow b := 1$
$r \rightarrow b := -1$
$t := r$

Az algoritmus helyessége hasonlóan gondolható meg, mint a beszúrás esetén. Lényeges különbség azonban, hogy ott minden beszúrást csak egyetlen kiegyensúlyozás követ, míg itt előfordulhat, hogy a minimális csúcs eltávolítása után, minden felette lévő szinten ki kell egyensúlyozni.

1.1. Feladat. *Mutassunk ilyen, legalább négy magasságú fát!*

1.2. Feladat. *Írjuk meg az $AVLremMin(t, minp, d)$ eljárás mintájára az $AVLremMax(t, maxp, d)$ eljárást és segédeljárásait!*

1.3. AVL fák: törlés

Bináris keresőfákra a $del(t, k)$ és a $delRoot(t)$ eljárások:

$del(\&t:Node^* ; k:\mathcal{T})$			
$t \neq \ominus$			
$k < t \rightarrow key$	$k > t \rightarrow key$	$k = t \rightarrow key$	SKIP
$del(t \rightarrow left, k)$	$del(t \rightarrow right, k)$	$delRoot(t)$	

$delRoot(\&t:Node^*)$		
$t \rightarrow left = \ominus$	$t \rightarrow right = \ominus$	$t \rightarrow left \neq \ominus \wedge t \rightarrow right \neq \ominus$
$p := t$	$p := t$	$remMin(t \rightarrow right, p)$
$t := p \rightarrow right$	$t := p \rightarrow left$	$p \rightarrow left := t \rightarrow left$
		$p \rightarrow right := t \rightarrow right$
delete p	delete p	delete $t ; t := p$

Ezeket fogjuk most az $AVLremMin(t, minp, d)$ eljárás mintájára kiegészíteni a d paraméterrel; majd a rekurzív töröl hívásokból, valamint az $AVLremMin$ eljárásból való visszatérés után megkérdezzük, csökkent-e az aktuális részfa magassága. Ha igen, az $AVLremMin(t, minp, d)$ eljárás mintájára módosítjuk a $t \rightarrow b$ értéket, ha kell, kiegyensúlyozunk, és ha kell, d -t beállítjuk.

AVLdel(&t:Node* ; k:ℤ ; &d:ℤ)					
$t \neq \emptyset$					
$k < t \rightarrow key$		$k > t \rightarrow key$		$k = t \rightarrow key$	$d := hamis$
AVLdel($t \rightarrow left, k, d$)		AVLdel($t \rightarrow right, k, d$)		AVLdelRoot(t, d)	
d		d			
leftSubTreeShrunk(t, d)	SKIP	rightSubTreeShrunk(t, d)	SKIP		

AVLdelRoot(&t:Node* ; &d:ℤ)			
$t \rightarrow left = \emptyset$	$t \rightarrow right = \emptyset$	$t \rightarrow left \neq \emptyset \wedge t \rightarrow right \neq \emptyset$	
$p := t$	$p := t$	rightSubTreeMinToRoot(t, d)	
$t := p \rightarrow right$	$t := p \rightarrow left$		
delete p	delete p	d	
$d := true$	$d := true$	rightSubTreeShrunk(t, d)	SKIP

rightSubTreeMinToRoot(&t:Node* ; &d:ℤ)	
AVLremMin($t \rightarrow right, p, d$)	
$p \rightarrow left := t \rightarrow left ; p \rightarrow right := t \rightarrow right ; p \rightarrow b := t \rightarrow b$	
delete $t ; t := p$	

Itt is lehetséges, hogy több szinten, a legrosszabb esetben akár minden szinten is ki kell egyensúlyozni. Mivel azonban egyetlen kiegyensúlyozás sem tartalmaz se rekurziót, se ciklust, és ezért konstans számú eljáráshívásból áll, ez sem itt, sem az AVLremMin eljárásnál nem befolyásolja a futási időnek az AVLinsert eljárásra is érvényes $MT(n) \in \Theta(\log n)$ (ahol $n = |t|$) nagyságrendjét.

1.3. Feladat. A $leftSubTreeShrunk(t, d)$ eljárás mintájára dolgozzuk ki a $rightSubTreeShrunk(t, d)$ eljárást, ennek segédeljárásait, és az ehhez szükséges kiegyensúlyozási sémát (megoldás: 5. ábra)! Mutassuk be az AVLdel(t, k) eljárás működését néhány példán! Mutassunk olyan, legalább négy magasságú fát és kulcsot, amelyre az AVLdel(t, k) eljárás minden, a fizikailag törölt csúcs feletti szinten kiegyensúlyozást végez!

1.4. Feladat. Írjuk meg az $AVLdel(t, k)$ eljárás egy olyan változatát, amely abban az esetben, ha a k kulcsot olyan belső csúcs tartalmazza, aminek két gyereke van, ezt a törlendő csúcsot a bal részfája legnagyobb kulcsú csúcsával helyettesíti!

1.4. Az AVL fák magassága*

Tétel: Tetszőleges n csúcsú nemüres AVL fa h magasságára:

$$\lfloor \log n \rfloor \leq h \leq 1,45 \log n$$

Bizonyítás:

Az $\lfloor \log n \rfloor \leq h$ egyenlőtlenség bizonyításához elég azt belátni, hogy ez tetszőleges nemüres bináris fára igaz. Egy tetszőleges bináris fa nulladik (gyöker) szintjén legfeljebb $2^0 = 1$ csúcs van, az első szintjén maximum $2^1 = 2$ csúcs, a második szintjén nem több, mint $2^2 = 4$ csúcs. Általában, ha az i -edik szinten 2^i csúcs van, akkor az $i + 1$ -edik szinten legfeljebb 2^{i+1} csúcs, hiszen minden csúcsnak maximum két gyereke van. Innét egy h mélységű bináris fa csúcsainak n számára $n \leq 2^0 + 2^1 + 2^2 + \dots + 2^h = 2^{h+1} - 1 < 2^{h+1}$.

Innét

$$\lfloor \log n \rfloor \leq \log n < \log 2^{h+1} = h + 1, \text{ amiből } \lfloor \log n \rfloor \leq h.$$

A $h \leq 1,45 \log n$ egyenlőtlenség bizonyításához elég azt belátni, hogy ez tetszőleges nemüres, kiegyensúlyozott bináris fára (KBF-re) igaz. Ehhez először meghatározzuk egy $h \geq 0$ magasságú (azaz nemüres) KBF csúcsainak minimális f_h számát. Nyilván $f_0 = 1$ és $f_1 = 2$, hiszen egy nulla magasságú KBF csak a gyökercsúcsból áll, az egy magasságú KBF-ek pedig ((B)G(J)), ((B)G), vagy (G(J)) alakúak.

Az is világos, hogy az $\langle f_0, f_1, f_2, \dots \rangle$ sorozat szigorúan monoton növekvő. (Ennek igazolásához vegyünk egy $i + 1$ magas, f_{i+1} csúcsú t KBF-et! Ekkor a t bal és jobb részfái közül az egyik magassága i . Legyen ez az f részfa! Jelölje most $s(b)$ egy tetszőleges b bináris fa csúcsainak számát! Akkor $f_{i+1} = s(t) > s(f) \geq f_i$.)

Ezután $h \geq 2$ esetén $f_h = 1 + f_{h-1} + f_{h-2}$. (Ha ugyanis t egy h magasságú, minimális, azaz f_h méretű KBF, akkor ennek bal és jobb részfáiban is, a részfák magasságához mérten a lehető legkevesebb csúcs van. Az egyik részfája kötelezően $h - 1$ magas, ebben tehát f_{h-1} csúcs van. Mivel t KBF, a másik részfája $h - 1$ vagy $h - 2$ magas. A másik részfában tehát f_{h-1} vagy f_{h-2} csúcs van, és $f_{h-2} < f_{h-1}$, tehát a másik részfában f_{h-2} csúcs van, és így $f_h = 1 + f_{h-1} + f_{h-2}$.)

A képlet emlékeztet a Fibonacci sorozatra:
 $F_0 = 0, F_1 = 1, F_h = F_{h-1} + F_{h-2}$, ha $h \geq 2$.

Megjegyzés: A h magasságú, és f_h méretű KBF-eket ezért *Fibonacci fák*-nak hívjuk. Az előbbiek szerint egy $h \geq 2$ magasságú Fibonacci fa mindig $(\varphi_{h-1} \text{ G } \varphi_{h-2})$ vagy $(\varphi_{h-2} \text{ G } \varphi_{h-1})$ alakú, ahol φ_{h-1} és φ_{h-2} $h-1$, illetve $h-2$ magasságú Fibonacci fák.

1.5. Feladat. *Rajzoljunk $h \in 0..4$ magasságú Fibonacci fákat!*

Megvizsgáljuk, van-e valami összefüggés az $\langle f_h : h \in \mathbb{N} \rangle$ és az $\langle F_h : h \in \mathbb{N} \rangle$ sorozatok között.

h	0	1	2	3	4	5	6	7	8	9
F_h	0	1	1	2	3	5	8	13	21	34
f_h	1	2	4	7	12	20	33			

A fenti táblázat alapján az első néhány értékre $f_h = F_{h+3} - 1$. Teljes indukcióval könnyen látható, hogy ez tetszőleges $h \geq 0$ egész számra igaz: Feltéve, hogy $0 \leq h \leq k \geq 1$ esetén igaz, $h = k+1$ -re:

$$f_h = f_{k+1} = 1 + f_k + f_{k-1} = 1 + F_{k+3} - 1 + F_{k+2} - 1 = F_{k+4} - 1 = F_{h+3} - 1.$$

Tudjuk, hogy

$$F_h = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^h - \left(\frac{1-\sqrt{5}}{2} \right)^h \right]$$

Az F_h -ra vonatkozó explicit képlet segítségével összefüggést adunk tetszőleges KBF n mérete és h magassága között.

$$\begin{aligned} n \geq f_h = F_{h+3} - 1 &= \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^{h+3} - \left(\frac{1-\sqrt{5}}{2} \right)^{h+3} \right] - 1 \geq \\ &\geq \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^{h+3} - \left| \left(\frac{1-\sqrt{5}}{2} \right)^{h+3} \right| \right] - 1 \end{aligned}$$

Mivel $2 < \sqrt{5} < 3$, azért $|1 - \sqrt{5}|/2 < 1$ és $\left| \left(\frac{1-\sqrt{5}}{2} \right)^{h+3} \right| < 1$. Eszerint

$$n > \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^{h+3} - 1 \right] - 1 = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^3 \left(\frac{1+\sqrt{5}}{2} \right)^h - \frac{1+\sqrt{5}}{\sqrt{5}}$$

Mivel

$$\frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^3 = \frac{1+3\sqrt{5}+3(\sqrt{5})^2+(\sqrt{5})^3}{8\sqrt{5}} = \frac{16+8\sqrt{5}}{8\sqrt{5}} = \frac{2+\sqrt{5}}{\sqrt{5}}$$

Ezt behelyettesítve az előbbi, n -re vonatkozó egyenlőtlenségbe:

$$n > \frac{2+\sqrt{5}}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^h - \frac{1+\sqrt{5}}{\sqrt{5}}$$

Az első együtthatót tagokra bontva, a disztributív szabállyal:

$$n > \left(\frac{1+\sqrt{5}}{2} \right)^h + \frac{2}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^h - \frac{1+\sqrt{5}}{\sqrt{5}}$$

Most

$$\frac{2}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^h - \frac{1+\sqrt{5}}{\sqrt{5}} \geq 0 \iff \left(\frac{1+\sqrt{5}}{2} \right)^h \geq \frac{1+\sqrt{5}}{2} \iff h \geq 1$$

Eszerint $h \geq 1$ esetén

$$n > \left(\frac{1+\sqrt{5}}{2} \right)^h$$

$$h = 0 \text{ -ra pedig } n = 1, \text{ és így } n = \left(\frac{1+\sqrt{5}}{2} \right)^h$$

A fentiekből tetszőleges, nemüres KBF n méretére és h magasságára

$$n \geq \left(\frac{1+\sqrt{5}}{2} \right)^h$$

Innét, ha vesszük mindkét oldal kettes alapú logaritmusát, majd $\log \frac{1+\sqrt{5}}{2}$ -tel osztunk:

$$h \leq \frac{1}{\log \frac{1+\sqrt{5}}{2}} \log n$$

Mivel $1,44 < 1,44042 < \frac{1}{\log \frac{1+\sqrt{5}}{2}} < 1,4404201 < 1,45$, azért tetszőleges, nemüres KBF n méretére és h magasságára

$$h \leq 1,45 \log n$$

2. Általános fák

Az általános fák esetében, a bináris fákkal összehasonlítva, egy csúcshoz tetszőlegesen sok gyereke lehet. Itt azonban, tetszőleges csúcshoz tartozó részfák száma pontosan egyenlő a gyerekek számával, azaz nem tartoznak hozzá üres részfák. Ha a gyerekek sorrendje lényeges, akkor *rendezett fákról* beszélünk.

Általános fákkal modellezhetjük például a számítógépünkben a mappák hierarchiáját, a programok blokkstruktúráját, a függvénykifejezéseket, a családakat és bármelyik hierarchikus struktúrát.

Vegyük észre, hogy ugyan minden konkrét általános fában van az egy csúcshoz tartozó gyerekek számára valamilyen r felső korlát, de ettől a fa még nem tekinthető r -árisnak, mert ez a korlát nem abszolút: a fa tetszőleges csúcsa gyerekeinek száma tetszőlegesen növelhető. Másrészt, mivel itt nem értelmezzük az *üres részfák* fogalmát, ez mégsem általánosítása az r -áris fa fogalmának. Értelmezzük azonban a gyökércsúcs (nincs szülője) és a levélcsúcs (nincs gyereke) fogalmát, azaz továbbra is gyökeres fákról beszélünk.²

Az általános fák természetes ábrázolási módja a *bináris láncolt* reprezentáció. Itt egy csúcs szerkezete a következő (ahol most *child1* az első gyereke, *sibling* pedig a következő testvérre mutat). A $*p$ csúcs akkor levél, ha $p \rightarrow \text{child1} = \emptyset$; és a $*p$ csúcs akkor utolsó testvér, ha $p \rightarrow \text{sibling} = \emptyset$.

Node
+ <i>child1, sibling</i> : Node* // <i>child1</i> : első gyerek; <i>sibling</i> : következő testvér
+ <i>key</i> : T // T ismert típus
+ Node() { <i>child1</i> := <i>sibling</i> := $\emptyset$ } // egycsúcsú fát képez belőle
+ Node($x:T$) { <i>child1</i> := <i>sibling</i> := $\emptyset$; <i>key</i> := x }

Természetesen itt is lehet *szülő* pointer, ami a gyerekek közös szülőjére mutat vissza.

2.1. Feladat. *Próbáljunk megadni az általános fákra másfajta láncolt reprezentációkat is! Hasonlítsuk össze az általunk adott ábrázolásokat és a bináris láncolt reprezentációt, memóriaigény és rugalmasság szempontjából!*

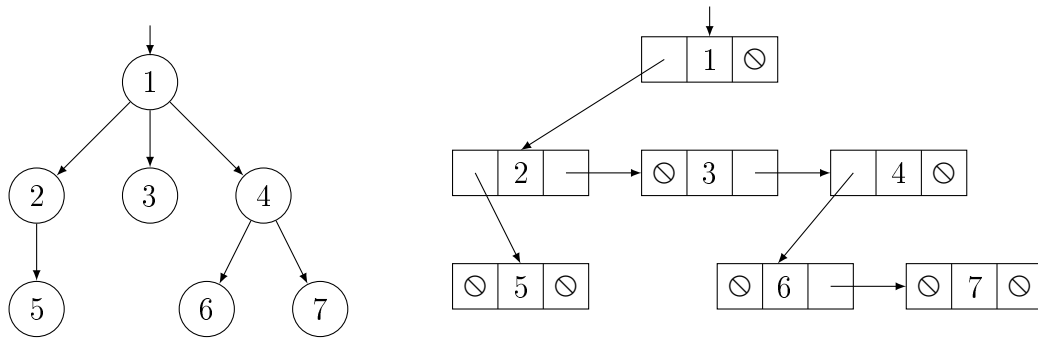
A szöveges (zárójeles) reprezentációban az általános fáknál a gyökeret előre szokás venni. Így egy nemüres fa általános alakja $(G \ t_1 \dots t_n)$, ahol G a gyökércsúcs tartalma, $t_1 \dots t_n$ pedig a részfák.

Így pl. az $\{ 1 \ [\ 2 \ (5) \] \ (3) \ [\ 4 \ (6) \ (7) \] \}$ általános fában az 1 van a gyökérben, a gyerekei a 2, a 3 és a 4 kulcsú csúcsok, a hozzájuk tartozó

²Ellentétben az ún. *szabad fákkal*, amik irányítatlan, körmentes gráfok.

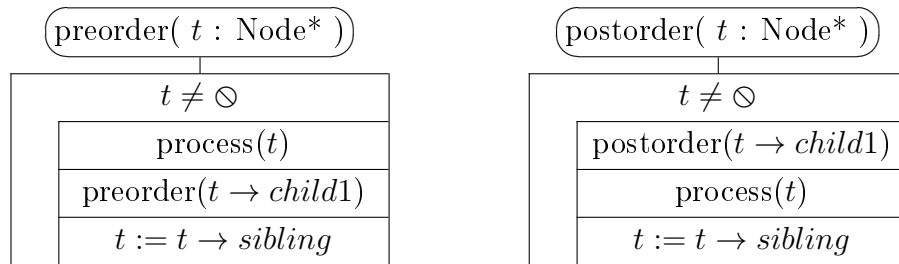
részfák pedig sorban a $[2 (5)]$, a (3) és a $[4 (6) (7)]$. A fa levelei az 5, a 3, a 6 és a 7 kulcsú csúcsok (7. ábra).

2.2. Feladat. Írjunk programot, ami kiír egy binárisan láncolt fát a fenti zárójeles alakban! Írjunk olyat is, ami egy szövegfájlból visszaolvassa! Készítsük el a kiíró és a visszaolvasó programokat az általunk kidolgozott alternatív láncolt ábrázolásokra is! (A visszaolvasó programokat általában nehezebb megírni.)



7. ábra. Az $\{ 1 [2 (5)] (3) [4 (6) (7)] \}$ általános fa absztrakt szerkezete (balra) és bináris reprezentációja (jobbra).

Az általános fa preorder bejárása³ a $child1 \sim left$ és $sibling \sim right$ megfeleltetéssel a bináris reprezentáció preorder bejárását igényli. Az általános fa postorder bejárásához⁴ azonban (az előbbi megfeleltetéssel) a bináris reprezentáció inorder bejárása szükséges.⁵



³először a gyökér, majd sorban a gyerekeihez tartozó részfák

⁴előbb sorban a gyökér gyerekeihez tartozó részfák, végül a gyökér

⁵A fájlrendszerekben általában preorder bejárás szerint keresünk, míg a függvénykifejezéseket (eltekintve most a lusta kiértékeléstől, aminek a tárgyalása túl messzire vezetne) postorder bejárással értékeljük ki.

2.3. Feladat. *Lássuk be a fenti bejárások helyességét! (Ötlet: A testvérek listájának mérete szerinti teljes indukció pl. célravezető.) Lássuk be egy ellenpélda segítségével, hogy az általános fa inorder bejárása⁶ nem szimulálható a bináris reprezentáció egyik nevezetes bejárásával⁷ sem! Írjuk meg a bináris láncolt reprezentációra az általános fa inorder bejárását és szintfolytonos bejárását is!*

2.4. Feladat. *Írjuk meg a fenti bejárásokat az általunk adott alternatív láncolt reprezentációkra is! Írjunk olyan programokat, amelyek képesek tetszőleges általános fa egy adott reprezentációjából egy adott másik ábrázolású másolatot készíteni!*

3. B+ fák és műveleteik

<http://aszt.inf.elte.hu/~asvanyi/ad/ad2jegyzet/B+ fa.pdf>

⁶A $(G \ t_1 \dots t_n)$ fára $n > 0$ esetén előbb t_1 -et járja be, majd feldolgozza G -t, aztán bejárja sorban a $t_2 \dots t_n$ részfákat; $n = 0$ esetén csak feldolgozza G -t.

⁷preorder, inorder, postorder, szintfolytonos